

Chapter 17.2

Fast RCNN Architecture

2015

தமிழில்

Less than 20 Mins

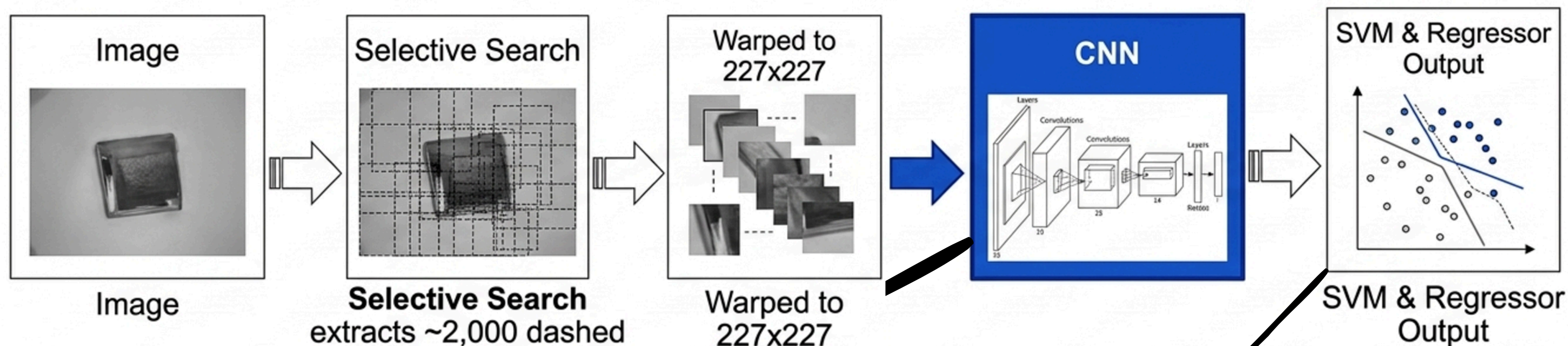
AI COACH JOHN

CEO & FOUNDER

Agenda at a glance

1. **Architectural Shift** from RCNN
2. Where it is applied? (***Real-Time Usecases***)
3. **BottleNeck Changed** (Limitations)
4. Takeaway

IMPORTANT



Why CNN here?

1. CNN **was NOT used for classification** here.
2. It was only used to extract important visual features.

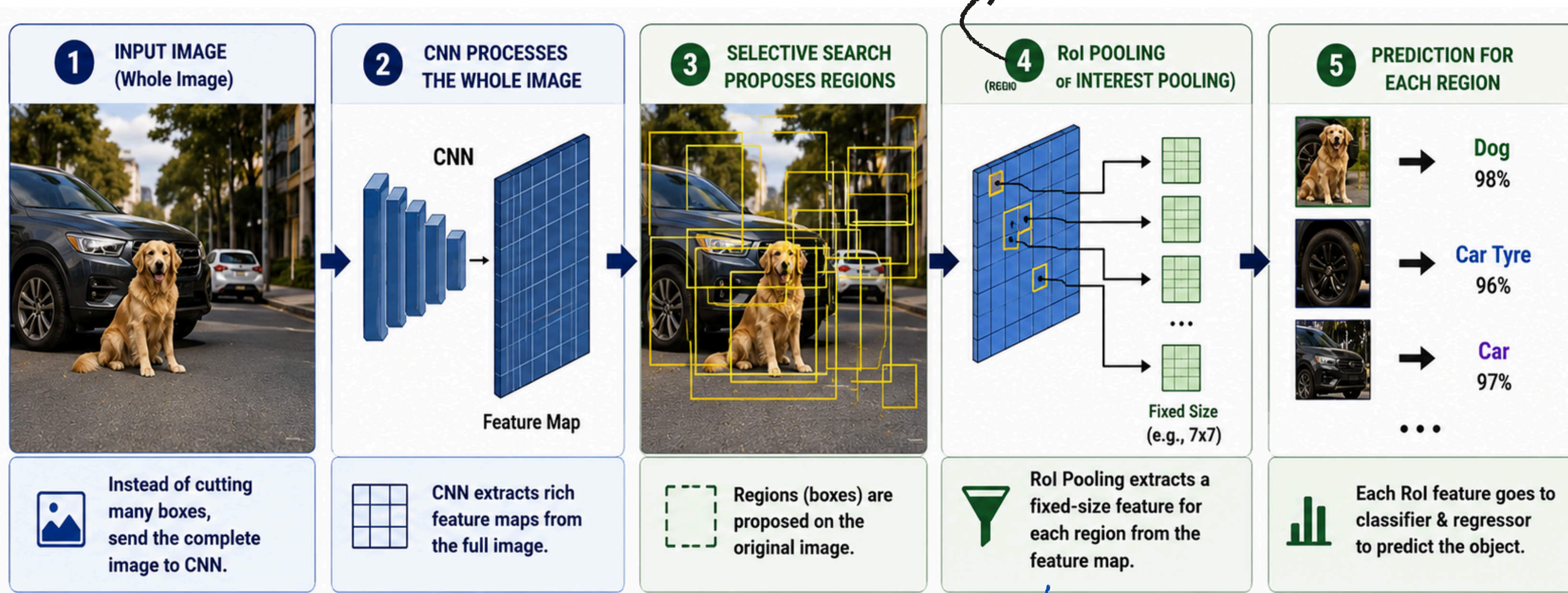
Why SVM & Regressor here?

1. **Why SVM?**
Those extracted numerical features were then given to SVM, because classical ML models can work only with structured data.
2. **Why Regressor?**
Corrects the bounding box coordinates

Point 1

Architectural Shift from RCNN

Architectural Shift from RCNN



What it does is, it takes those ~2000 region proposals and directly projects them onto the generated feature map.

By doing this, it does not recalculate from the raw pixels again. Instead, it extracts the data only from the coordinates. This completely skips running the CNN 2000 individual times.

Because of this architectural change, the processing time for one image was reduced from **47 seconds to just 2 seconds**. It was a massive breakthrough

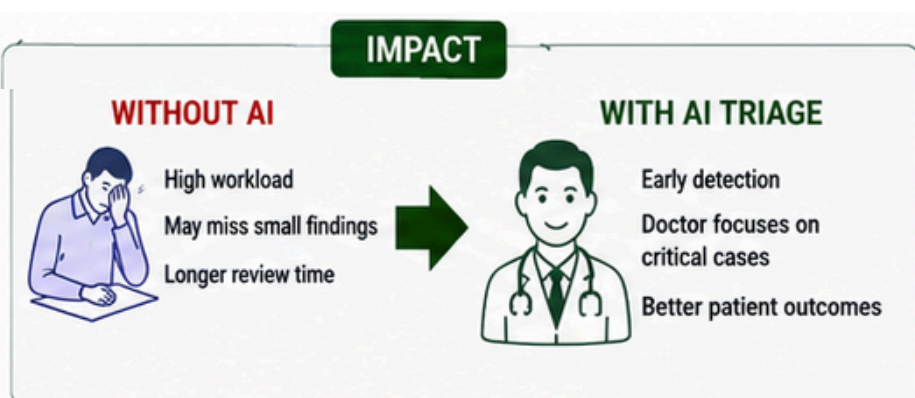
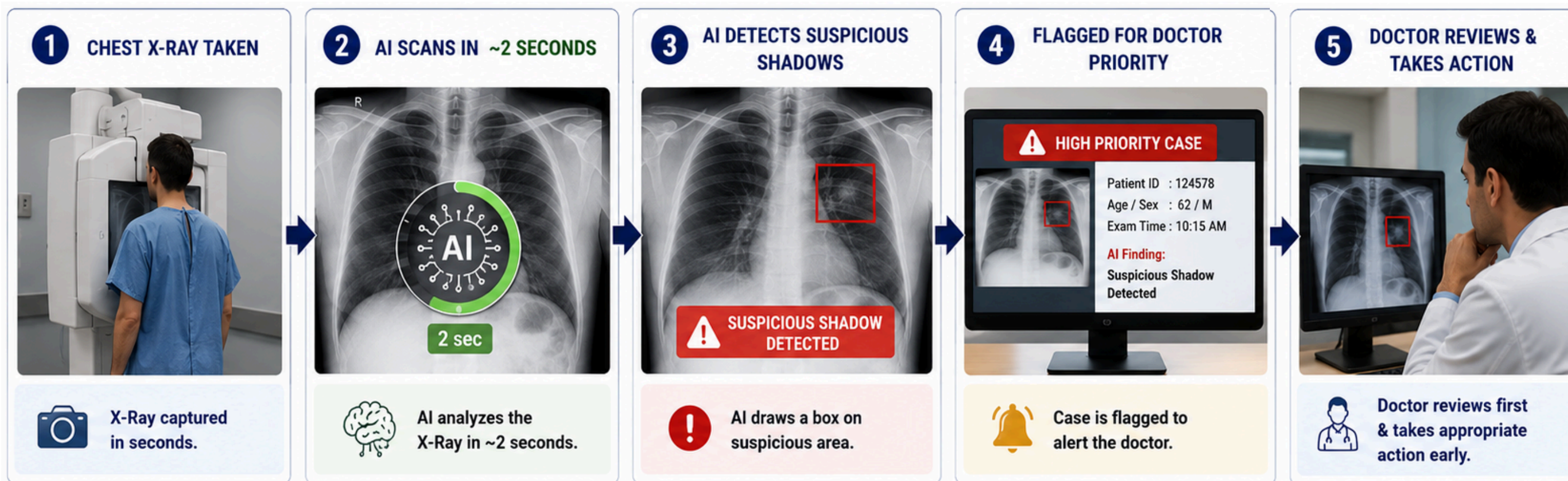
"Fast RCNN"

Point 2

Where it is applied?
(Real-Time Usecases)

1. Chest X-Ray → AI Powered Medical TRIAGE for...

Fast. Accurate. Helps Doctors Focus on What Matters.



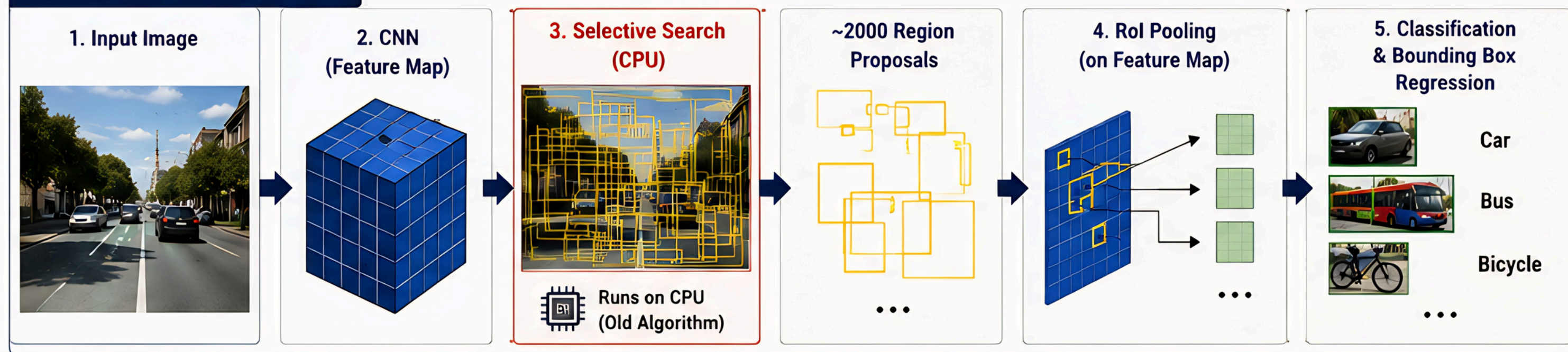
Point 3

Now bottleneck changed
Limitations

Bottleneck Shift → Deep Learning became ***fast***.

But proposal generation was ***still slow***.

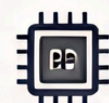
FAST R-CNN PIPELINE



THE BOTTLENECK



Fast R-CNN made Deep Learning math very fast (using GPU).



But ~2000 region boxes are generated by Selective Search (non-neural algorithm).



Selective Search runs on CPU.



CPU becomes the bottleneck. Even if GPU is very fast, it has to wait for ~2000 boxes.

Like a traffic jam in the system!



Data keeps shifting between CPU & GPU. This causes high latency.



⇒ To reduce latency, **keep the complete process in GPU!**

Point 4

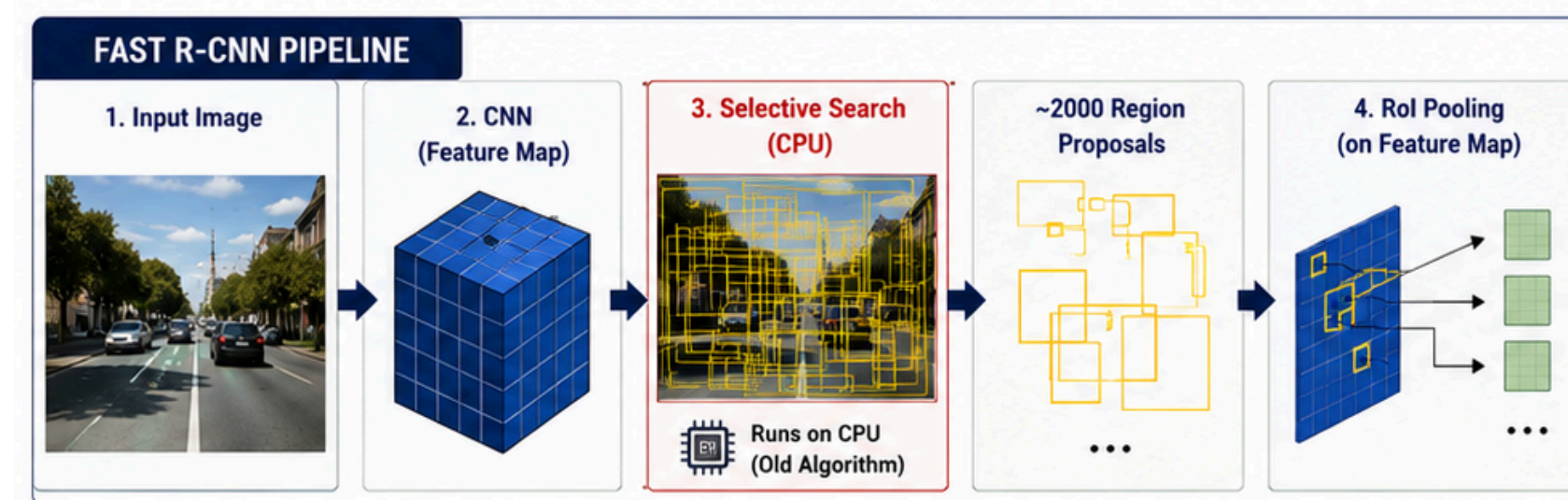
Takeaway

Key Takeaway Point

- **R-CNN** solved → **Object Localization**
- **Fast R-CNN** solved → **Repeated CNN computation**

But Still...

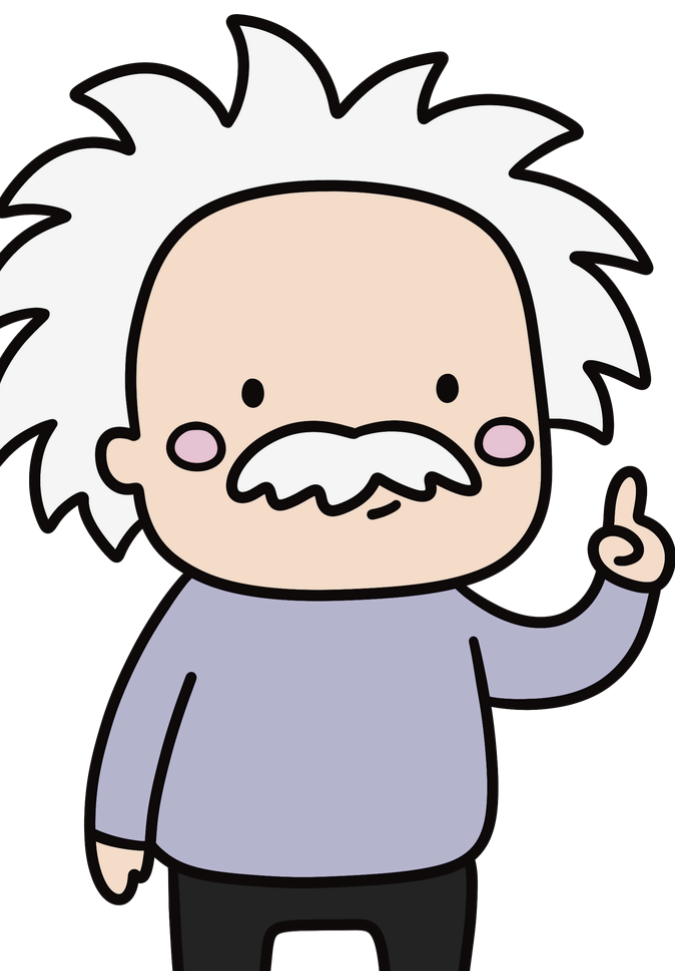
- 1. Selective Search on CPU
- 2. GPU waiting problem



Next question

*"If GPU already understands the image, why not let the **neural network itself** "propose the boxes " and keeps all calculations inside GPU itself?"*

And this gave birth to
"Faster RCNN"



Summary

Overall Summary

1. Architectural Shift from RCNN
2. Where it is applied? (*Real-Time Usecases*)
3. Bottleneck Changed (Limitations)
4. Takeaway



RCNN → Fast RCNN

Massive Breakthrough

"47 Sec/Image" → 2 Sec/Image